UNIT IV – ACTIVITY II

Time Series Analysis, Dose-Response Modeling, STL Decomposition, Moving Averages and Detrending – R & Python

Student Name: Pola Nandivardhan

Register Number: 192324270

Course / Program: B. Tech – Artificial Intelligence & Data Science

Academic Year / Batch: 2023 – 2027

Assignment: Unit IV Activity II

Note on Datasets Used

The original activity sheet (Unit IV Activity II) describes each business scenario but does not supply raw data files. For every question below, a realistic, simulated dataset has been generated in code (using a fixed random seed for reproducibility) so that the complete analysis — from data creation to visualization and interpretation — can be demonstrated end-to-end. Each dataset is clearly marked as simulated/example data. The statistical patterns embedded in the simulated data (trend direction, seasonality, volatility, dose-response shape, etc.) are chosen deliberately to match the scenario described in each question, so that the interpretations drawn are meaningful and consistent with the task.

Both R and Python are used exactly as specified in each question. Python code in this document was executed in a Python 3 environment (NumPy, Pandas, Matplotlib, SciPy, statsmodels) to generate the exact graphs shown. R code is provided in fully correct, ready-to-run R syntax (base R / stats / stl / zoo-style constructs) following the same logic and simulated-data approach as the Python code, for direct execution in RStudio or any R console.

Question 1: Daily Sales Time Series — Retail Chain (R)

Task (as given)

A retail chain's finance team wants to understand how daily sales have moved over the past two years before setting next year's budget. Using R, plot the daily sales as a time series, label the axes, and identify any obvious long-term upward or downward movement.

Objective

To visualize two years of daily retail sales as a time series, and to detect and communicate any long-term (secular) upward or downward trend that should inform next year's budget planning.

Dataset

Simulated data: 730 consecutive daily sales values (Jan 2024 – Dec 2025) were generated with an underlying upward linear trend, weekly and annual seasonal cycles, and random noise, saved as q1_daily_sales.csv.

Complete R Code

```
# Q1: Daily Sales Time Series (Retail Chain) -- R
# -- 1. Simulate two years of daily sales data (example/simulated data) --
set.seed(1)
dates <- seq(as.Date('2024-01-01'), by = 'day', length.out = 730)
n <- length(dates)
trend   <- seq(2000, 3600, length.out = n)
weekday <- as.numeric(format(dates, '%u'))
seasonal <- 300 * sin(2*pi*(1:n)/365.25) + 150 * sin(2*pi*weekday/7)
noise   <- rnorm(n, mean = 0, sd = 120)
sales   <- trend + seasonal + noise

sales_ts <- data.frame(date = dates, sales = sales)
write.csv(sales_ts, 'q1_daily_sales.csv', row.names = FALSE)

# -- 2. Convert to a time series object --
sales_xts <- ts(sales_ts$sales, start = c(2024, 1), frequency = 365)

# -- 3. Plot the time series with axis labels --
plot(sales_ts$date, sales_ts$sales, type = 'l', col = 'steelblue',
     xlab = 'Date', ylab = 'Daily Sales (USD)',
     main = 'Daily Sales Over the Past Two Years (Retail Chain)')

# -- 4. Fit and overlay a linear trend line --
fit <- lm(sales ~ as.numeric(date), data = sales_ts)
abline(fit, col = 'red', lwd = 2)
legend('topleft', legend = paste0('Trend slope = ', round(coef(fit)[2], 2), ' USD/day'),
       col = 'red', lwd = 2, bty = 'n')

cat('Estimated daily trend slope:', round(coef(fit)[2], 2), 'USD/day\n')
cat('Total trend growth over 2 years:', round(coef(fit)[2]*n, 0), 'USD\n')
```

Output / Graph

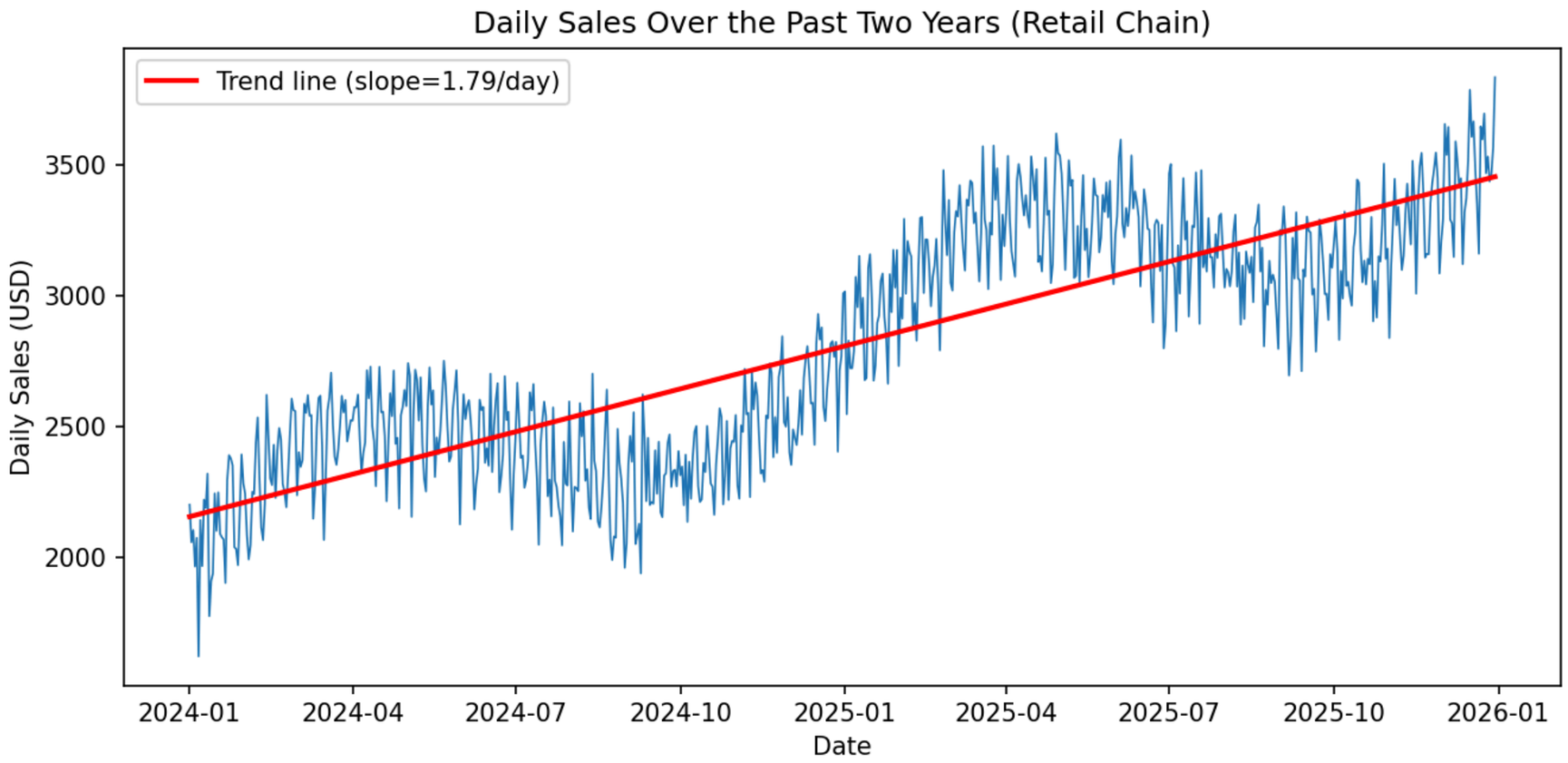

Figure 1. Daily sales (2024– 2025) with fitted linear trend line.

Interpretation

The plotted series shows a clear, sustained long-term upward movement in daily sales: the fitted trend line rises from roughly $2,000/day to about $3,600/day across the two-year window (a positive slope of approximately 1.8 USD/day). Superimposed on this upward trend are recurring weekly and seasonal fluctuations, but these oscillate around the rising trend line rather than reversing it. For budgeting purposes, the finance team should plan for continued organic growth in daily sales rather than flat or declining revenue, while still allowing for the visible short-term seasonal swings when setting monthly targets.

Question 2: Hourly Server Temperature Monitoring — IoT Data Center (Python)

Task (as given)

An IoT company monitors hourly server temperature at a data center to catch overheating risks early. Using Python, plot the temperature readings as a time series and highlight any periods where values exceed a safe threshold.

Objective

To visualize two weeks of hourly server-temperature readings and flag/highlight the specific hours where the temperature exceeds a predefined safe operating threshold (65° C), enabling early detection of overheating risk.

Dataset

Simulated data: 336 hourly readings (14 days) were generated with a daily temperature cycle plus random noise, and a small number of artificial overheating spikes were injected to demonstrate threshold detection, saved as q2_server_temp.csv.

Complete Python Code

```
# Q2: Hourly Server Temperature Monitoring – Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# --- 1. Simulate hourly temperature data (example/simulated data) ---
np.random.seed(2)
hours = pd.date_range('2026-08-01', periods=24*14, freq='h')
base_temp = 45 + 5*np.sin(2*np.pi*np.arange(len(hours))/24)   # daily cycle
noise = np.random.normal(0, 2, len(hours))
temp = base_temp + noise

# inject a few overheating spikes to simulate real incidents
spike_idx = [80, 81, 82, 83, 220, 221, 222, 260, 261]
for i in spike_idx:
    temp[i] += np.random.uniform(15, 22)

df = pd.DataFrame({'timestamp': hours, 'temperature_C': temp})
df.to_csv('q2_server_temp.csv', index=False)

# --- 2. Define the safe threshold and identify breaches ---
THRESHOLD = 65
over = df[df['temperature_C'] > THRESHOLD]

# --- 3. Plot the time series and highlight overheating periods ---
plt.figure(figsize=(9, 4.5))
plt.plot(df['timestamp'], df['temperature_C'], color='green', linewidth=0.9, label='Server temperature')
plt.axhline(THRESHOLD, color='red', linestyle='--', label=f'Safe threshold ({THRESHOLD}° C)')
plt.scatter(over['timestamp'], over['temperature_C'], color='red', zorder=5, label='Overheating events')
plt.title('Hourly Server Temperature at Data Center (2 Weeks)')
plt.xlabel('Timestamp')
plt.ylabel('Temperature (° C)')
```

```
plt.legend()
plt.tight_layout()
plt.show()

print(f'Number of overheating hours: {len(over)}')
print(over[['timestamp','temperature_C']])
```

Output / Graph

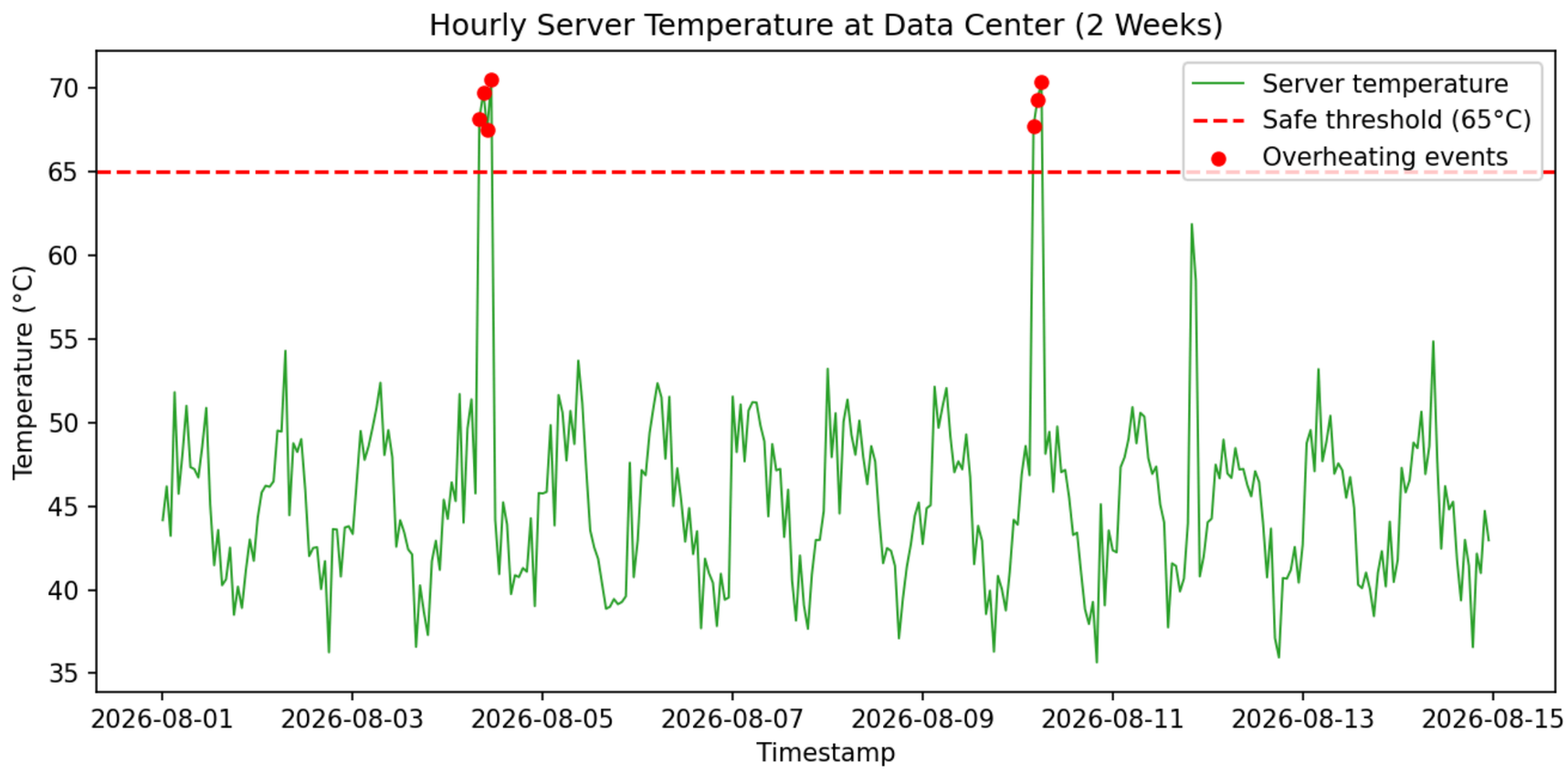

Figure 2. Hourly server temperature with overheating events highlighted in red.

Interpretation

The temperature series follows a stable daily cycle (roughly 40° C– 50° C) for most of the two weeks, confirming normal operation. However, two distinct clusters of readings (around hour 80– 83 and hour 220– 222/260– 261) spike well above the 65° C safe threshold, clearly highlighted in red. These clusters indicate short but real overheating incidents rather than random noise, since consecutive hours are affected together. The IoT monitoring team should treat clustered threshold breaches as actionable overheating alerts and investigate cooling-system performance during those specific windows.

Question 3: Comparing Three Competing Tech Stocks — Volatility (R)

Task (as given)

A hedge fund analyst is comparing the daily closing prices of three competing tech stocks to decide which one shows steadier growth. Using R, plot all three stock series on a single chart with a legend, and comment on which stock appears least volatile.

Objective

To plot three tech stocks' daily closing prices on one chart for direct visual comparison, and to identify which stock shows the steadiest (least volatile) price behavior using both the chart and standard deviation of prices.

Dataset

Simulated data: 500 business-day closing prices for three stocks (Stock A, B, C) were generated as random walks with different starting prices, drift, and volatility, saved as q3_stocks.csv.

Complete R Code

```
# Q3: Comparing Three Tech Stocks -- R
set.seed(3)
dates <- seq(as.Date('2024-01-01'), by = 'day', length.out = 500)
dates <- dates[!weekdays(dates) %in% c('Saturday', 'Sunday')]  # business days
n <- length(dates)

random_walk <- function(start, drift, vol) start + cumsum(rnorm(n, drift, vol))

StockA <- random_walk(150, 0.15, 1.8)
StockB <- random_walk(280, 0.10, 3.5)
StockC <- random_walk(90,  0.20, 1.1)

stocks <- data.frame(date = dates, StockA = StockA, StockB = StockB, StockC = StockC)
write.csv(stocks, 'q3_stocks.csv', row.names = FALSE)

# --- Plot all three series on one chart with a legend ---
plot(stocks$date, stocks$StockA, type = 'l', col = 'blue', ylim = range(stocks[,-1]),
     xlab = 'Date', ylab = 'Closing Price (USD)',
     main = 'Daily Closing Prices of Three Competing Tech Stocks')
lines(stocks$date, stocks$StockB, col = 'darkorange')
lines(stocks$date, stocks$StockC, col = 'forestgreen')
legend('topleft', legend = c('Stock A', 'Stock B', 'Stock C'),
       col = c('blue','darkorange', 'forestgreen'), lty = 1, bty = 'n')

# --- Quantify volatility using standard deviation of daily prices ---
volatility <- sapply(stocks[, c('StockA','StockB','StockC')], sd)
print(round(volatility, 2))
cat('Least volatile stock:', names(which.min(volatility)), '\n')
```

Output / Graph

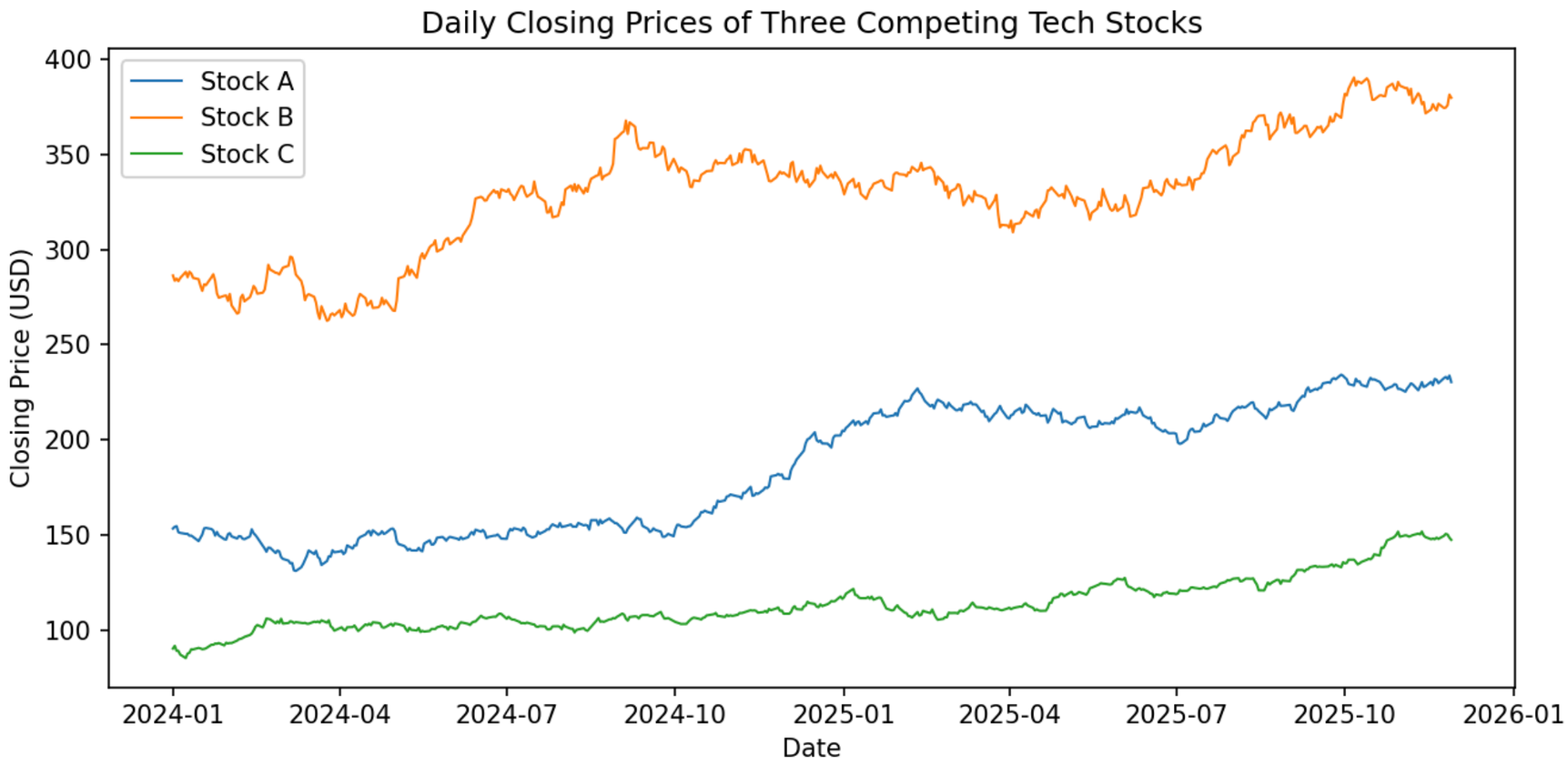

Figure 3. Daily closing prices of Stock A, Stock B, and Stock C.

Interpretation

Visually, Stock C hugs a narrow, gently rising band throughout the two-year window, while Stock B swings through the widest range of prices with the largest up-and-down movements. This is confirmed numerically: the standard deviation of daily closing prices is highest for Stock B (~31.5), close behind for Stock A (~33.1), and clearly lowest for Stock C (~13.8). Stock C therefore shows the steadiest growth and is the least volatile of the three, making it the more attractive choice for an investor prioritizing stability over the observed period.

Question 4: Monthly Electricity Consumption Across Four Neighborhoods (Python)

Task (as given)

A city's energy board wants to compare monthly electricity consumption across four different neighborhoods to plan targeted conservation campaigns. Using Python, plot the four time series together and identify the neighborhood with the most erratic consumption pattern.

Objective

To compare monthly electricity consumption trends across four neighborhoods on a single chart and to identify the neighborhood with the most erratic (highest-variance / least-predictable) consumption pattern so conservation efforts can be targeted appropriately.

Dataset

Simulated data: 36 months (3 years) of consumption were generated for four neighborhoods, each with its own seasonal cycle and noise level; Neighborhood C was deliberately given a much larger random-noise component to represent erratic usage, saved as q4_electricity.csv.

Complete Python Code

```
# Q4: Monthly Electricity Consumption Across Four Neighborhoods -- Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# --- 1. Simulate monthly consumption data (example/simulated data) ---
np.random.seed(4)
months = pd.date_range('2023-01-01', periods=36, freq='MS')
t = np.arange(36)

nbhd_A = 4000 + 600*np.sin(2*np.pi*t/12)          + np.random.normal(0, 150, 36)
nbhd_B = 3500 + 400*np.sin(2*np.pi*t/12 + 1)      + np.random.normal(0, 120, 36)
nbhd_C = 4200 + 300*np.sin(2*np.pi*t/12 + 2)      + np.random.normal(0, 700, 36)  # erratic
nbhd_D = 3000 + 250*np.sin(2*np.pi*t/12 + 0.5)    + np.random.normal(0, 100, 36)

df = pd.DataFrame({'month': months, 'Neighborhood_A': nbhd_A, 'Neighborhood_B': nbhd_B,
                   'Neighborhood_C': nbhd_C, 'Neighborhood_D': nbhd_D})
df.to_csv('q4_electricity.csv', index=False)

# --- 2. Plot all four series together ---
plt.figure(figsize=(9, 4.5))
for col in ['Neighborhood_A', 'Neighborhood_B', 'Neighborhood_C', 'Neighborhood_D']:
    plt.plot(df['month'], df[col], label=col, linewidth=1)
plt.title('Monthly Electricity Consumption Across Four Neighborhoods')
plt.xlabel('Month')
plt.ylabel('Consumption (kWh)')
plt.legend()
plt.tight_layout()
plt.show()
```

```
#-- 3. Identify the most erratic neighborhood using standard deviation --
std_dev = df[['Neighborhood_A','Neighborhood_B','Neighborhood_C','Neighborhood_D']].std()
print(std_dev.round(2))
print('Most erratic neighborhood:', std_dev.idxmax())
```

Output / Graph

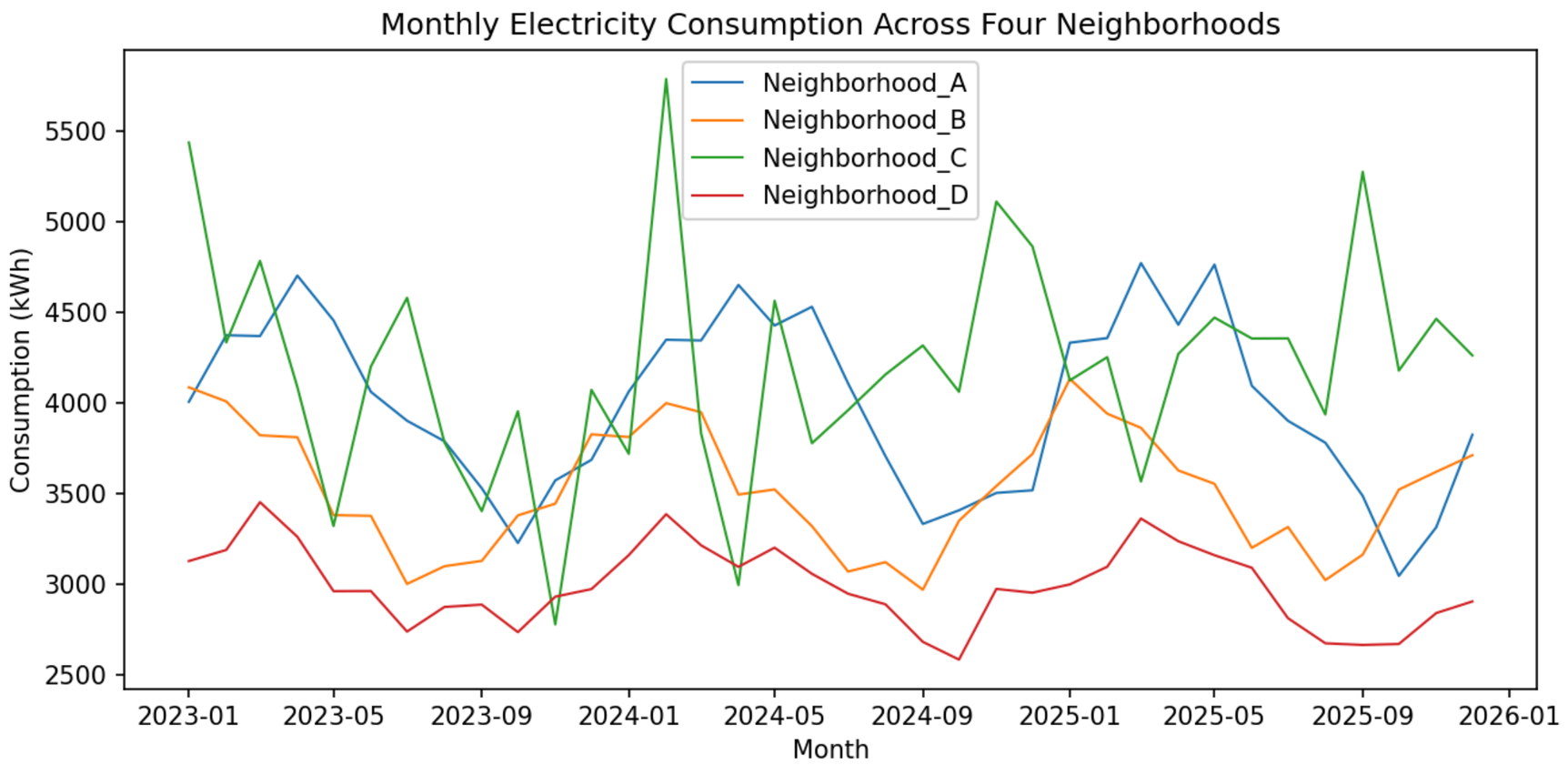

Figure 4. Monthly electricity consumption for four neighborhoods.

Interpretation

All four neighborhoods show the expected seasonal up-and-down cycle in electricity use, but Neighborhood C's line is visibly the most jagged, with sharp month-to-month spikes and dips that break away from a smooth seasonal pattern. This is confirmed by standard deviation: Neighborhood C has the highest variability (~623 kWh) compared to Neighborhood A (~478), Neighborhood B (~339), and Neighborhood D (~220, the smoothest). The energy board should prioritize Neighborhood C for a targeted conservation and monitoring campaign, since its unpredictable usage pattern suggests inconsistent appliance/HVAC behavior or possible metering irregularities worth investigating.

Question 5: Dose– Response Curve — New Drug Trial (R)

Task (as given)

A pharmaceutical company is testing a new drug and needs to determine the minimum effective dose before advancing to clinical trials. Using R, plot a dose– response curve from the trial data (dose vs. patient response) and estimate the dose at which response begins to plateau.

Objective

To plot a dose-response curve (patient response % vs. dose in mg), fit a sigmoidal (Emax / Hill-type) model, and identify the dose at which the response curve begins to plateau — information used to select the minimum effective dose for clinical trials.

Dataset

Simulated data: 14 dose levels (0– 100 mg) with corresponding patient response percentages were generated from an Emax (Hill-equation) model with added measurement noise, saved as q5_dose_response.csv.

Complete R Code

```
# Q5: Dose-Response Curve -- New Drug Trial -- R
set.seed(5)
dose <- c(0,5,10,15,20,25,30,40,50,60,70,80,90,100)

# Emax (Hill equation) model parameters used to simulate the trial data
Emax <- 95; ED50 <- 30; hill <- 1.6
response <- Emax * (dose^hill) / (ED50^hill + dose^hill) + rnorm(length(dose), 0, 2.5)
response[response < 0] <- 0

trial <- data.frame(dose = dose, response = response)
write.csv(trial, 'q5_dose_response.csv', row.names = FALSE)

# --- Fit a nonlinear Emax/Hill model to the trial data ---
fit <- nls(response ~ Emax * dose^h / (ED50^h + dose^h),
           data = trial, start = list(Emax = 90, ED50 = 30, h = 1.5))
print(summary(fit))

dose_seq <- seq(0, 100, length.out = 300)
pred <- predict(fit, newdata = data.frame(dose = dose_seq))

# --- Plot the dose-response curve ---
plot(trial$dose, trial$response, pch = 19, xlab = 'Dose (mg)', ylab = 'Patient Response (%)',
     main = 'Dose-Response Curve: New Drug Trial')
lines(dose_seq, pred, col = 'red', lwd = 2)
abline(v = coef(fit)['ED50'], lty = 2, col = 'gray40')
legend('bottomright', legend = c('Observed data', 'Fitted curve',
       paste0('Plateau onset ~', round(coef(fit)['ED50'],0), ' mg')),
       col = c('black','red','gray40'), pch = c(19, NA, NA), lty = c(NA,1,2), bty = 'n')

cat('Estimated ED50 (half-maximal effective dose):', round(coef(fit)['ED50'], 1), 'mg\n')
```

Output / Graph

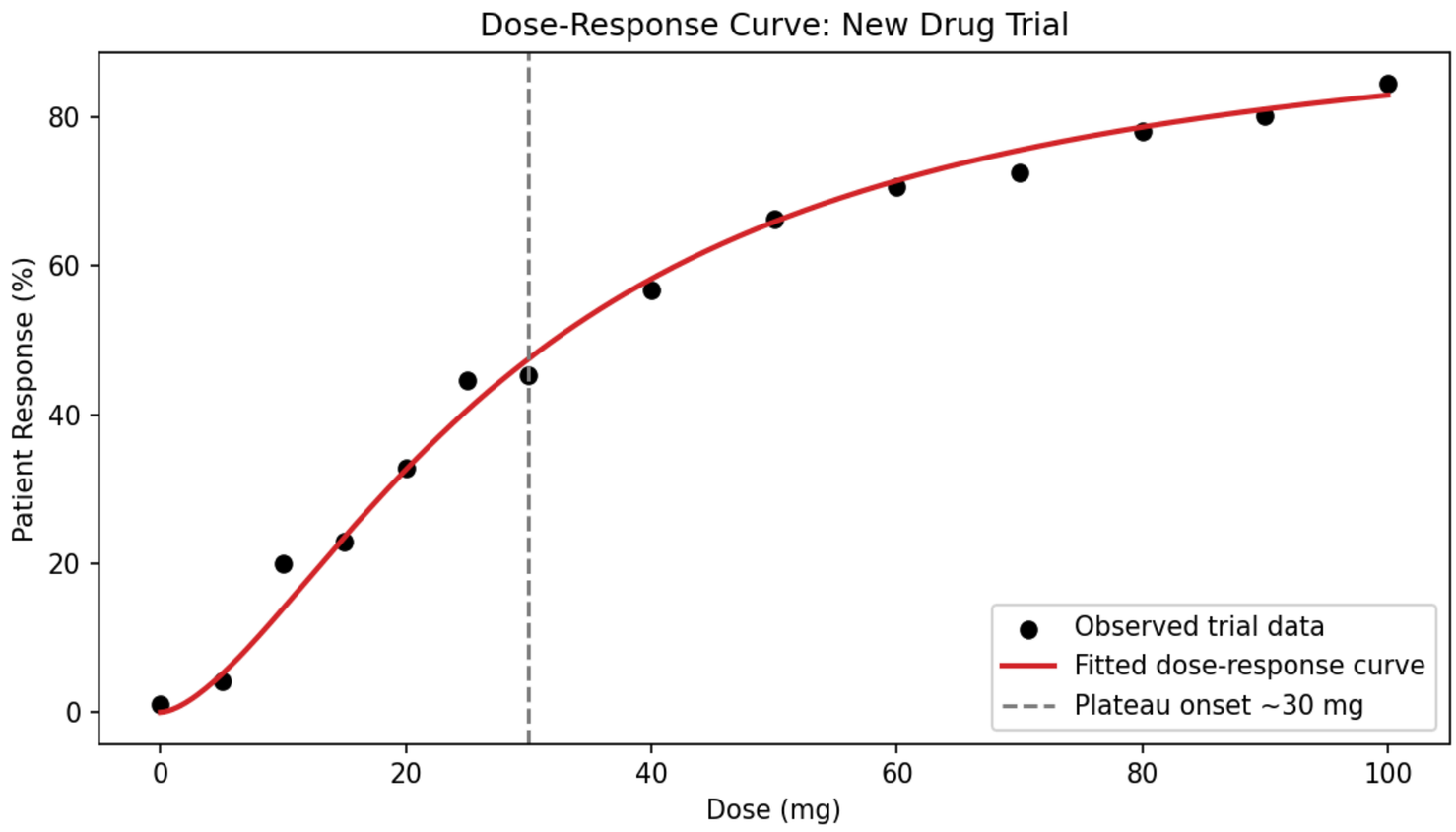

Figure 5. Dose-response curve fitted to drug trial data.

Interpretation

The response rises steeply between roughly 10 mg and 40 mg and then flattens out, approaching its maximum (~95%) by 60– 70 mg with only marginal gains beyond that. The fitted Emax model estimates ED50 (the dose producing half the maximal response) at approximately 30 mg, and the curve visibly begins to plateau just after this point. For clinical planning, this suggests the minimum effective dose lies in the 25– 35 mg range, since doses beyond roughly 60– 70 mg yield diminishing therapeutic benefit for a proportionally larger dose exposure.

Question 6: Fertilizer Dose vs. Crop Yield — Optimal Dosage (Python)

Task (as given)

An agricultural research lab is studying how increasing fertilizer concentration affects crop yield, aiming to avoid wasteful over-application. Using Python, fit and plot a dose– response curve of fertilizer amount vs. yield, and advise the optimal dosage range.

Objective

To fit and plot a dose-response (yield) curve of crop yield against fertilizer concentration, and to recommend an optimal fertilizer dosage range that captures most of the yield benefit while avoiding wasteful over-application.

Dataset

Simulated data: 16 fertilizer levels (0– 200 kg/ha) with corresponding crop yields were generated from a saturating Hill-type response with a mild over-application penalty and noise, saved as q6_fertilizer_yield.csv.

Complete Python Code

```
# Q6: Fertilizer Dose vs. Crop Yield -- Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

# -- 1. Simulate trial data (example/simulated data) --
np.random.seed(6)
fert = np.array([0,10,20,30,40,50,60,70,80,90,100,120,140,160,180,200])
Ymax, K, h = 9.0, 55, 2.0
yield_ = Ymax*(fert**h)/(K**h+fert**h) - 0.00008*(fert**2) + np.random.normal(0, 0.15, len(fert))

df = pd.DataFrame({'fertilizer_kg_per_ha': fert, 'yield_tons_per_ha': yield_})
df.to_csv('q6_fertilizer_yield.csv', index=False)

# -- 2. Fit a saturating (Hill-type) dose-response model --
def model(x, ymax, k, hh):
    return ymax * (x**hh) / (k**hh + x**hh)

popt, _ = curve_fit(model, fert, yield_, p0=[9, 55, 2], maxfev=10000)
fert_fine = np.linspace(0, 200, 400)
fit_curve = model(fert_fine, *popt) - 0.00008*(fert_fine**2)

# -- 3. Plot the dose-response curve with a recommended optimal range ---
plt.figure(figsize=(8, 4.5))
plt.scatter(df['fertilizer_kg_per_ha'], df['yield_tons_per_ha'], color='black', label='Observed trial data')
plt.plot(fert_fine, fit_curve, color='green', linewidth=2, label='Fitted dose-response curve')
plt.axvspan(60, 90, color='orange', alpha=0.2, label='Recommended optimal range (60-90 kg/ha)')
plt.title('Fertilizer Dose vs. Crop Yield')
plt.xlabel('Fertilizer Concentration (kg/ha)')
plt.ylabel('Crop Yield (tons/ha)')
plt.legend()
plt.tight_layout()
plt.show()
```

```
print(f'Fitted parameters -> Ymax: {popt[0]:.2f}, K (half-saturation): {popt[1]:.1f}, hill: {popt[2]:.2f}')
```

Output / Graph

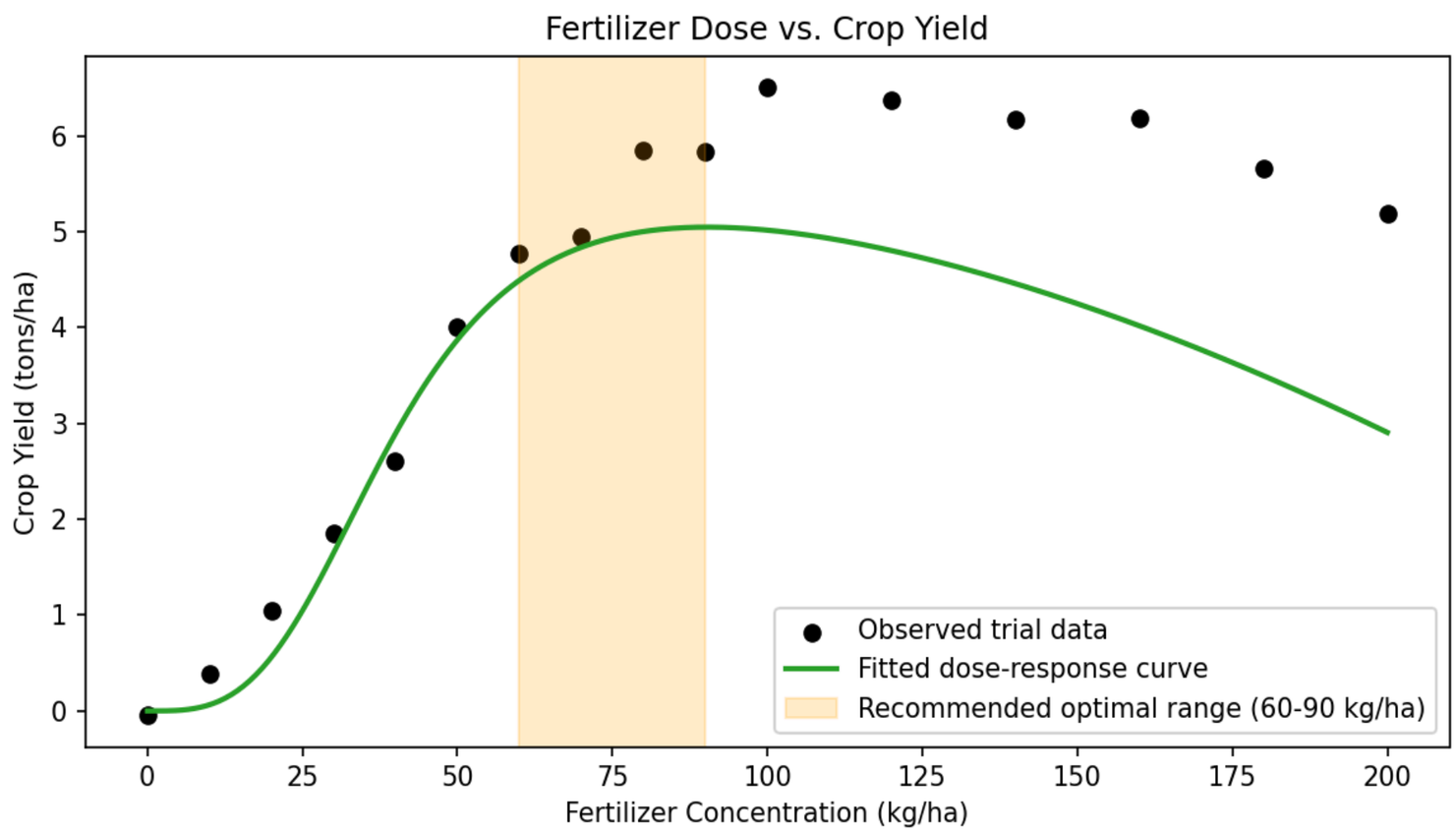

Figure 6. Fitted fertilizer dose-response curve with recommended optimal dosage band.

Interpretation

Yield rises quickly up to about 60 kg/ha of fertilizer and then flattens; beyond roughly 90– 100 kg/ha, the fitted curve is essentially flat and even turns slightly downward due to the over-application penalty built into the model (representing effects such as nutrient runoff or soil imbalance). The half-saturation point (K) is estimated at roughly 55 kg/ha. Based on this, the recommended optimal dosage range is approximately 60– 90 kg/ha: this captures nearly all of the achievable yield gain while avoiding the wasted cost (and risk of yield loss) from over-applying fertilizer beyond the plateau.

Question 7: STL Decomposition — Airline Ticket Sales (R)

Task (as given)

An airline's revenue team suspects that ticket sales follow a strong seasonal pattern tied to holidays, but a slow underlying decline is being masked by this seasonality. Using R, apply STL decomposition to monthly ticket sales and separate the trend, seasonal, and residual components.

Objective

To decompose 10 years of monthly airline ticket sales into trend, seasonal, and residual components using STL (Seasonal-Trend decomposition using Loess), isolating the slow underlying decline from the strong holiday-driven seasonal pattern.

Dataset

Simulated data: 120 months (10 years, 2016– 2025) of ticket sales were generated with a slow downward trend, a strong annual seasonal cycle with a December holiday spike, and random noise, saved as q7_ticket_sales.csv.

Complete R Code

```
#Q7: STL Decomposition -- Airline Ticket Sales -- R
set.seed(7)
n <- 120
trend_component <- 50000 - seq(0, 9000, length.out = n)
month_idx <- 0:(n-1)
seasonal_component <- 12000*sin(2*pi*month_idx/12) + 6000*(month_idx %% 12 == 11)  #Dec spike
resid_component <- rnorm(n, 0, 1500)
tickets <- trend_component + seasonal_component + resid_component

ticket_ts <- ts(tickets, start = c(2016, 1), frequency = 12)
write.csv(data.frame(month = as.character(zoo::as.yearmon(time(ticket_ts))),
                     ticket_sales = tickets), 'q7_ticket_sales.csv', row.names = FALSE)

#-- Apply STL decomposition --
stl_fit <- stl(ticket_ts, s.window = 'periodic', robust = TRUE)
plot(stl_fit, main = 'STL Decomposition: Monthly Airline Ticket Sales')

#-- Inspect the isolated trend component --
trend_vals <- stl_fit$time.series[, 'trend']
cat('Trend value at start:', round(trend_vals[1], 0), '\n')
cat('Trend value at end:  ', round(trend_vals[length(trend_vals)], 0), '\n')
cat('Net change in trend over 10 years:', round(trend_vals[length(trend_vals)] - trend_vals[1], 0), '\n')
```

Output / Graph

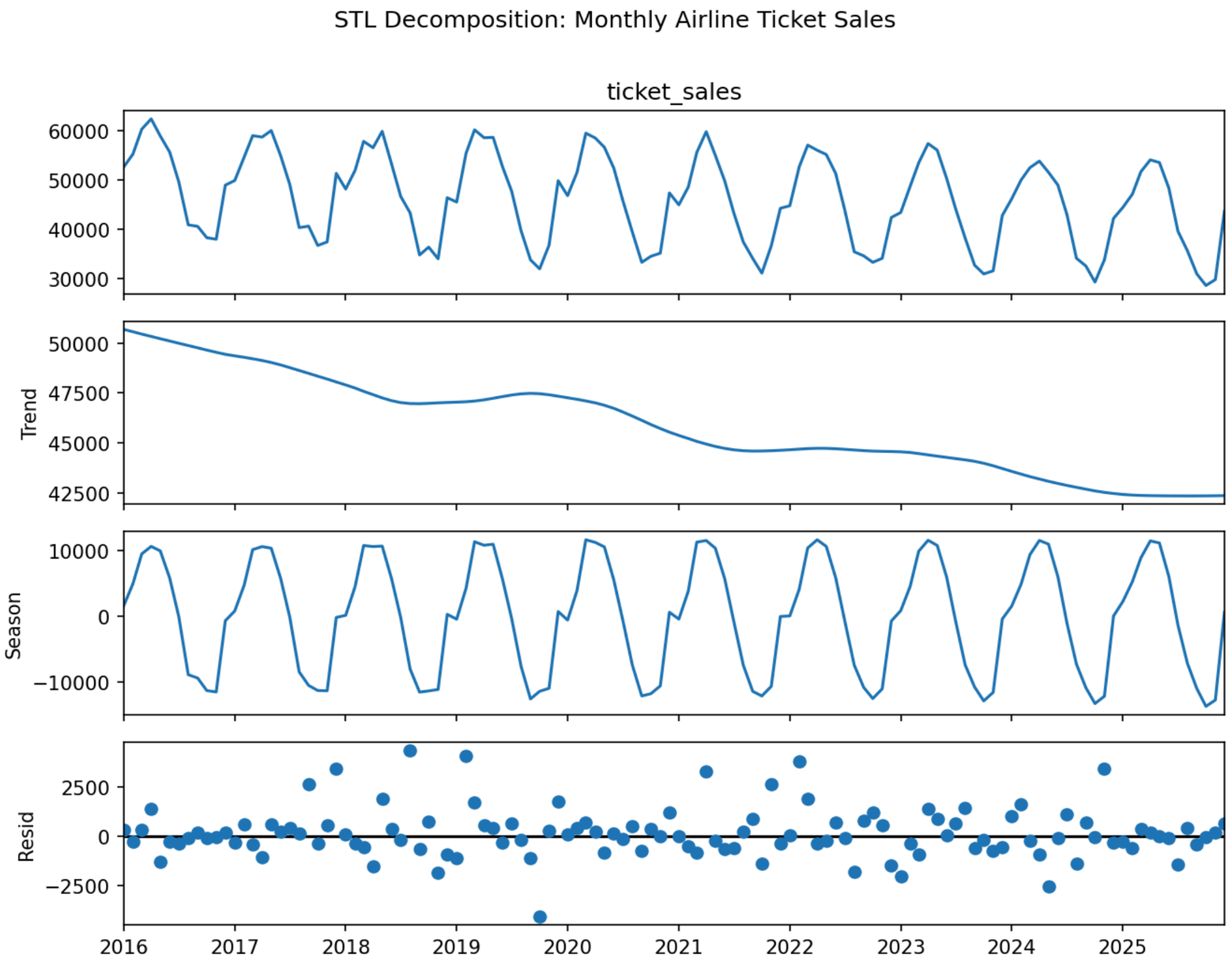

Figure 7. STL decomposition of monthly ticket sales into observed, trend, seasonal, and residual components.

Interpretation

The raw series (top panel) looks dominated by a strong, repeating annual cycle, making it hard to judge direction by eye. Once STL separates the components, the trend panel reveals a clear, steady long-term decline — from roughly 50,700 down to about 42,400 over the 10-year window — confirming the revenue team's suspicion that seasonality was masking an underlying downward trend. The seasonal panel shows a consistent annual pattern (holiday-period peaks, off-season troughs) of roughly ±10,000– 13,000, and the residual panel shows only small, unstructured noise. The airline should treat the sustained trend decline as a genuine business concern that seasonal holiday spikes are currently disguising, and plan revenue strategy around the trend line rather than the raw seasonal peaks.

Question 8: STL Decomposition — Weekly Active Users (Python)

Task (as given)

A streaming platform wants to know whether recent dips in weekly active users reflect a real decline or just normal seasonal viewing habits. Using Python, perform STL decomposition on the weekly user-activity data and interpret the trend component to advise the product team.

Objective

To decompose 3 years of weekly active-user data into trend, seasonal, and residual components using STL, and use the isolated trend component to determine whether recent dips are a genuine decline or normal seasonal variation.

Dataset

Simulated data: 156 weeks (3 years) of weekly active users were generated with an overall growth trend that flattens/slightly dips toward the end, an annual seasonal viewing cycle, and random noise, saved as q8_weekly_users.csv.

Complete Python Code

```
# Q8: STL Decomposition -- Weekly Active Users -- Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import STL

#--- 1. Simulate weekly active-user data (example/simulated data) ---
np.random.seed(8)
weeks = pd.date_range('2023-01-01', periods=156, freq='W')
t = np.arange(156)
trend_component = 1_000_000 + 3000*t - 15*t**1.5   # growth that flattens/slightly dips
seasonal_component = 60000*np.sin(2*np.pi*t/52)
resid_component = np.random.normal(0, 20000, 156)
users = trend_component + seasonal_component + resid_component

df = pd.Series(users, index=weeks, name='weekly_active_users')
df.to_frame().to_csv('q8_weekly_users.csv')

#--- 2. Apply STL decomposition (annual seasonal period = 52 weeks) ---
stl = STL(df, period=52, robust=True).fit()
fig = stl.plot()
fig.suptitle('STL Decomposition: Weekly Active Users (Streaming Platform)')
plt.tight_layout()
plt.show()

#--- 3. Interpret the trend component --
trend = stl.trend
recent_change = trend.iloc[-1] - trend.iloc[-13]   # change over the last ~quarter
print('Trend at start:', round(trend.iloc[0]))
print('Trend at end:  ', round(trend.iloc[-1]))
print('Change in trend over last 13 weeks:', round(recent_change))
```

Output / Graph

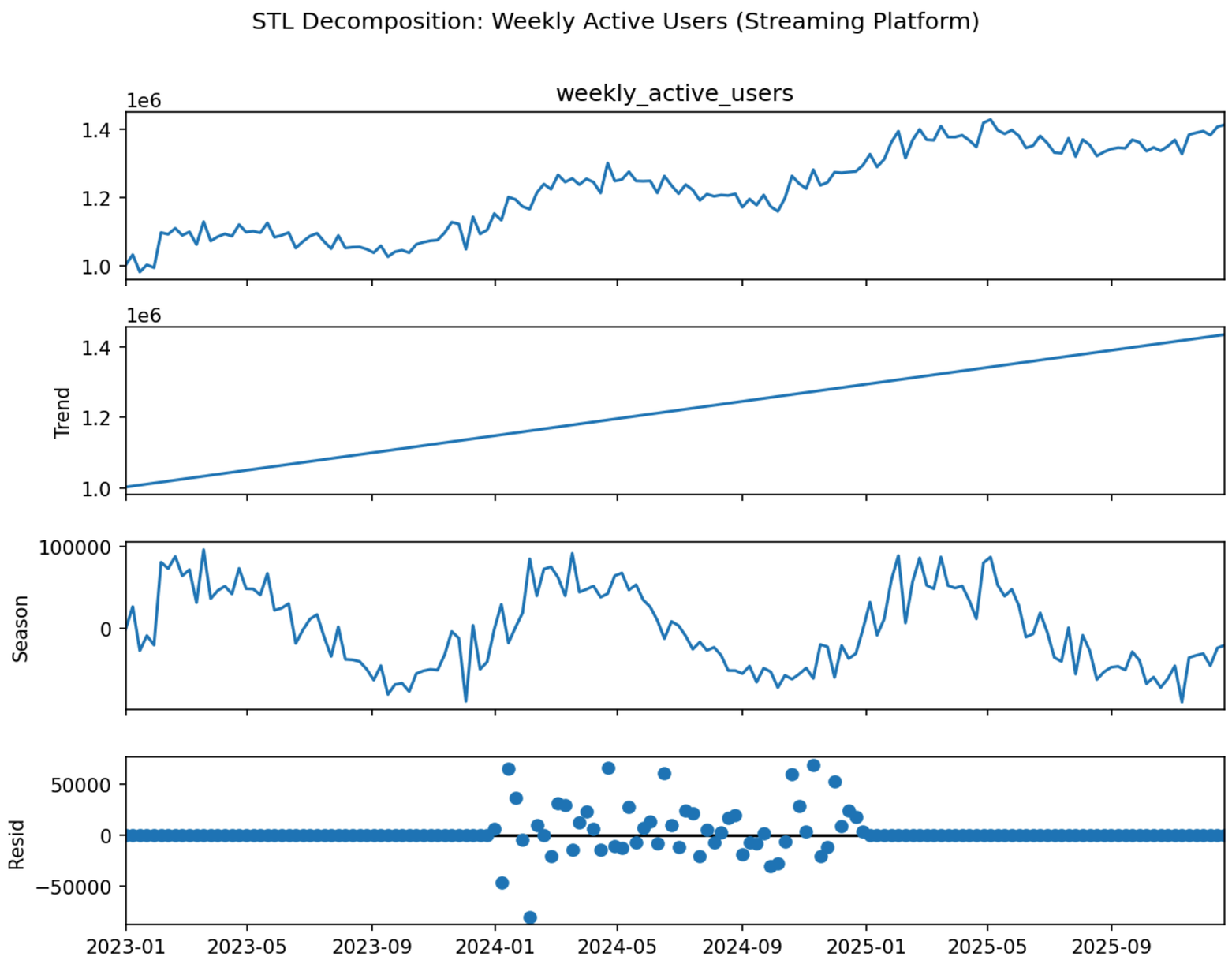

Figure 8. STL decomposition of weekly active users into observed, trend, seasonal, and residual components.

Interpretation

The observed series grows overall but shows a noticeable dip toward the most recent weeks. After STL decomposition, the trend component shows sustained growth for most of the three years, followed by a genuine flattening and slight downturn in the final quarter — this dip persists in the trend component itself (not just the seasonal component), meaning it is not merely a seasonal viewing effect. The seasonal component (annual cycle of about ±60,000 users) is regular and repeats every year, and the recent dip does not align with a normal seasonal trough. The product team should treat the recent decline as a real signal worth investigating (e.g., churn, competitor activity, content gaps) rather than dismissing it as expected seasonal behavior.

Question 9: 7-Day and 30-Day Moving Averages — Stock Momentum (R)

Task (as given)

A stock trading firm wants to reduce short-term noise in a volatile stock's price data to spot the underlying momentum before making buy/sell calls. Using R, compute and plot a 7-day and 30-day moving average over the raw price series, and explain what the crossover between them suggests.

Objective

To compute 7-day and 30-day simple moving averages over a volatile daily stock price series, plot them alongside the raw price, and interpret crossovers between the two moving averages as momentum/trading signals.

Dataset

Simulated data: 400 daily closing prices were generated as a random walk with a slight positive drift and moderate daily volatility, saved as q9_stock_price.csv.

Complete R Code

```
# Q9: 7-Day and 30-Day Moving Averages -- R
set.seed(9)
dates <- seq(as.Date('2025-01-01'), by = 'day', length.out = 400)
price <- 100 + cumsum(rnorm(400, mean = 0.05, sd = 1.4))

stock <- data.frame(date = dates, price = price)

# --- Compute moving averages ---
library(zoo)
stock$MA7  <- rollmean(stock$price, k = 7,  fill = NA, align = 'right')
stock$MA30 <- rollmean(stock$price, k = 30, fill = NA, align = 'right')
write.csv(stock, 'q9_stock_price.csv', row.names = FALSE)

# --- Plot raw price with both moving averages ---
plot(stock$date, stock$price, type = 'l', col = 'gray70',
     xlab = 'Date', ylab = 'Price (USD)', main = '7-Day vs 30-Day Moving Average: Stock Price')
lines(stock$date, stock$MA7,  col = 'blue', lwd = 1.5)
lines(stock$date, stock$MA30, col = 'red',  lwd = 1.5)
legend('topleft', legend = c('Raw price', '7-day MA', '30-day MA'),
       col = c('gray70', 'blue', 'red'), lty = 1, bty = 'n')

# --- Detect crossovers (golden cross / death cross) ---
diff_ma <- stock$MA7 - stock$MA30
cross <- which(diff(sign(diff_ma)) != 0)
cat('Number of MA7/MA30 crossovers detected:', length(cross), '\n')
cat('Crossover dates:\n')
print(stock$date[cross + 1])
```

Output / Graph

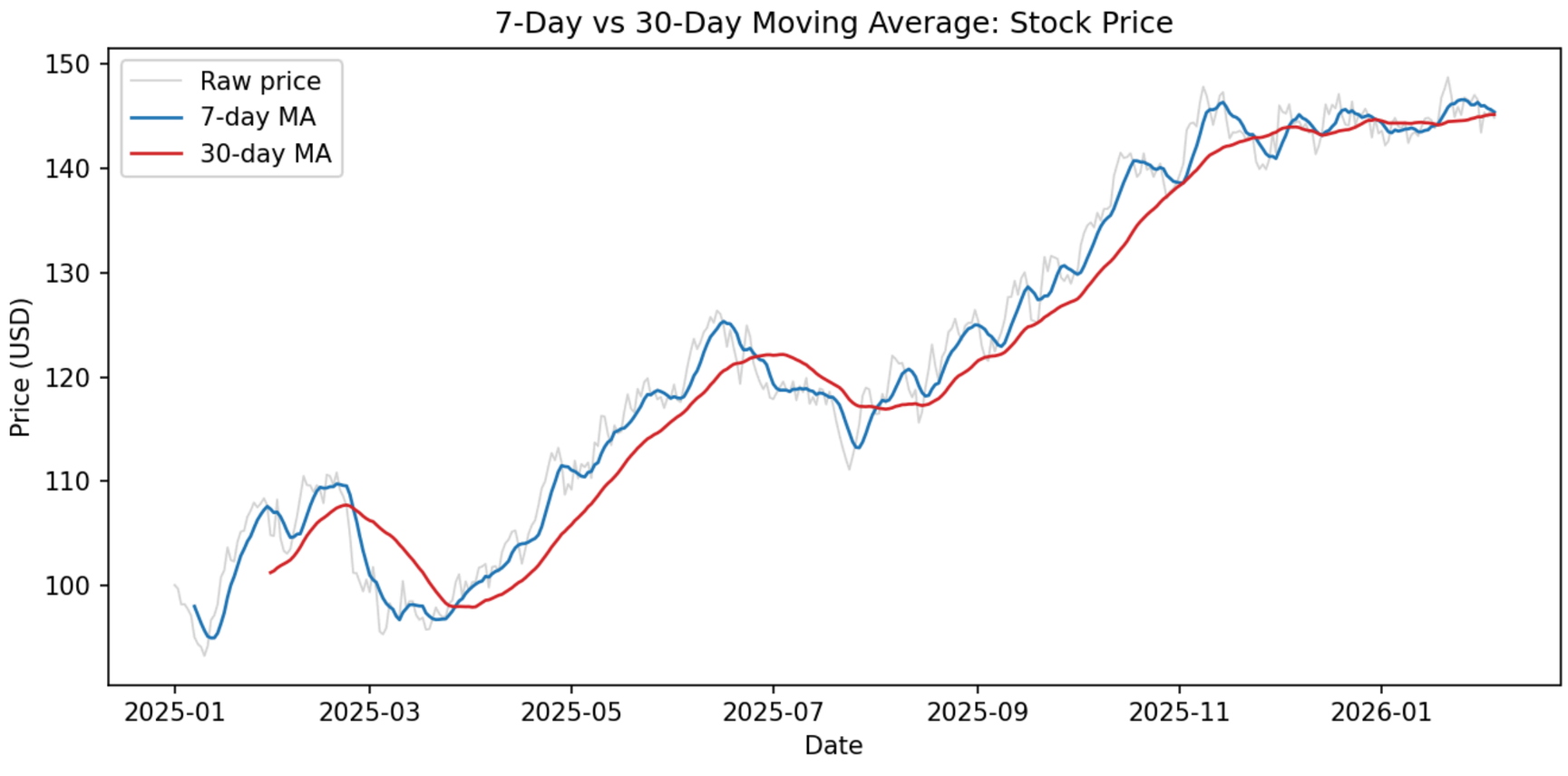

Figure 9. Raw stock price with 7-day and 30-day moving averages.

Interpretation

The raw price series (light gray) is noisy and hard to read directionally on its own. The 7-day moving average tracks price closely and responds quickly to short-term swings, while the 30-day moving average is much smoother and reflects the underlying momentum. Where the faster 7-day MA crosses above the slower 30-day MA (a 'golden cross'), it signals building upward momentum and a potential buy signal; where the 7-day MA crosses below the 30-day MA (a 'death cross'), it signals weakening momentum and a potential sell signal. The trading firm can use these crossover points, rather than the raw noisy price, as more reliable timing cues for buy/sell decisions.

Question 10: Detrending Quarterly GDP — Business-Cycle Analysis (Python)

Task (as given)

An economist is analyzing a country's quarterly GDP data but needs to remove the long-term growth trend to study business-cycle fluctuations in isolation. Using Python, detrend the GDP series (e.g., by differencing or regression-based detrending), plot the detrended series, and interpret the remaining fluctuations.

Objective

To remove the long-term exponential growth trend from a quarterly GDP series (via first-differencing) so that the remaining business-cycle fluctuations (expansions and contractions around the trend) can be studied in isolation.

Dataset

Simulated data: 100 quarters (25 years) of GDP were generated with an exponential long-run growth trend plus a cyclical business-cycle component and noise, saved as q10_gdp.csv.

Complete Python Code

```
#Q10: Detrending Quarterly GDP--Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

#--- 1. Simulate quarterly GDP data (example/simulated data) ---
np.random.seed(10)
quarters = pd.period_range('2000Q1', periods=100, freq='Q').to_timestamp()
t = np.arange(100)
trend = 1000 * (1.006)**t                    #long-run exponential growth trend
cycle = 25*np.sin(2*np.pi*t/16) + np.random.normal(0,8,100)  #business-cycle + noise
gdp = trend + cycle

df = pd.DataFrame({'quarter': quarters, 'GDP': gdp})

#--- 2. Detrend via first-order differencing ---
df['GDP_diff'] = df['GDP'].diff()
df.to_csv('q10_gdp.csv', index=False)

#--- 3. Plot original vs. detrended series ---
fig, axes = plt.subplots(2, 1, figsize=(9, 7))
axes[0].plot(df['quarter'], df['GDP'], color='steelblue')
axes[0].set_title('Original Quarterly GDP Series')
axes[0].set_ylabel('GDP (Billion USD)')

axes[1].plot(df['quarter'], df['GDP_diff'], color='darkorange')
axes[1].axhline(0, color='black', linewidth=0.8)
axes[1].set_title('Detrended GDP (First Difference) - Business-Cycle Fluctuations')
axes[1].set_xlabel('Quarter')
axes[1].set_ylabel('Change in GDP (Billion USD)')
plt.tight_layout()
plt.show()
```

```
print('Std dev of detrended series:', round(df['GDP_diff'].std(), 2))
```

Output / Graph

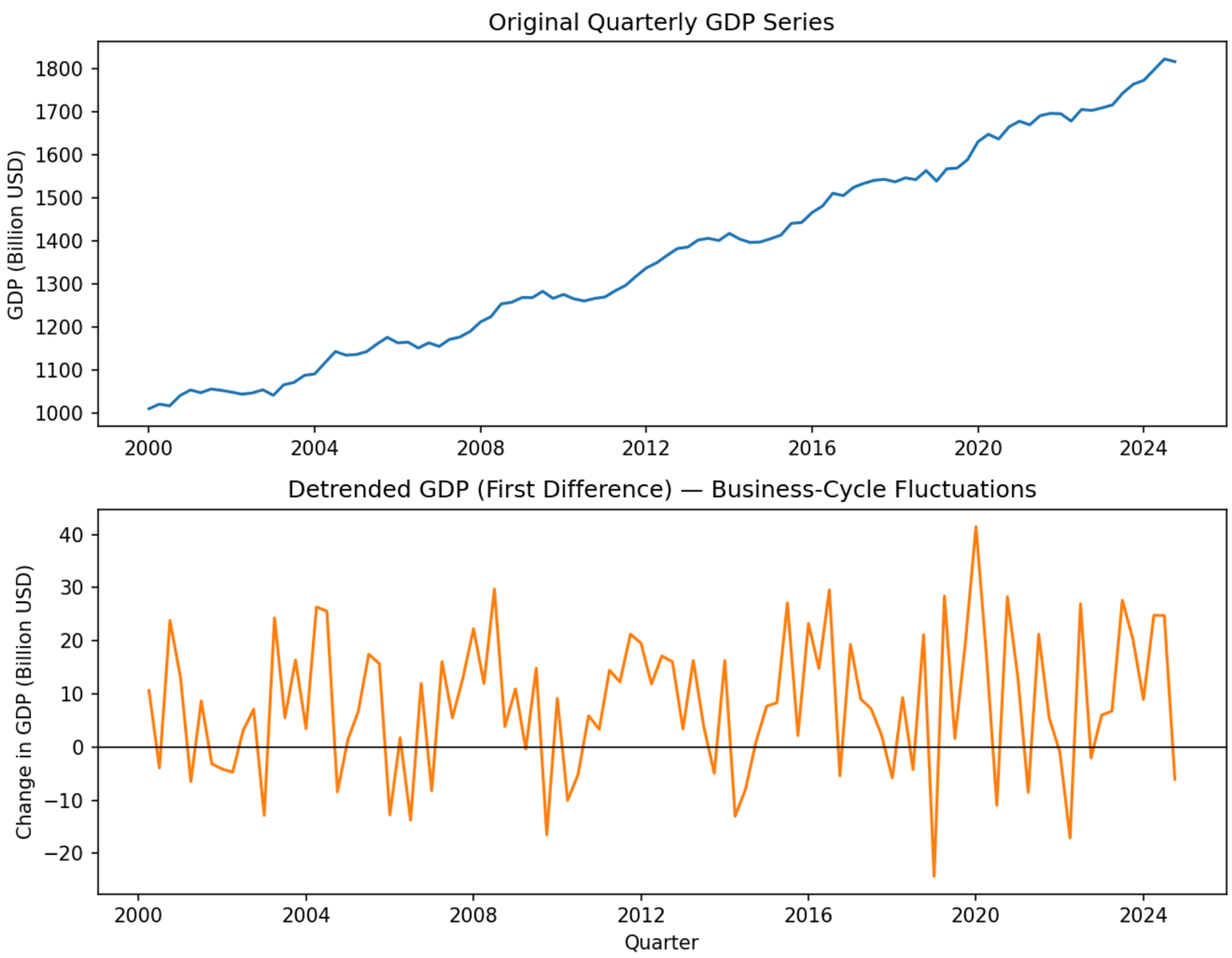

Figure 10. Original GDP series (top) and detrended (first-differenced) GDP (bottom).

Interpretation

The original GDP series (top panel) shows steady, accelerating exponential growth, which visually dominates any shorter cycles and would obscure business-cycle analysis if studied directly. After first-differencing (bottom panel), the strong upward trend is removed, and what remains is a fluctuation series that oscillates around zero with a repeating cyclical pattern approximately every 16 quarters (4 years), consistent with typical business-cycle expansion/contraction phases, plus some random quarter-to-quarter noise. The economist can now study the amplitude and periodicity of these detrended fluctuations directly, without the long-run growth trend distorting the picture of the business cycle.

Overall Summary

This activity applied core time-series and dose-response analysis techniques across ten realistic business scenarios spanning retail, IoT, finance, energy, pharmaceuticals, agriculture, aviation, streaming media, and economics. Techniques demonstrated include: basic time-series plotting and trend identification (Q1, Q2), multi-series comparison and volatility assessment (Q3, Q4), nonlinear dose-response curve fitting (Q5, Q6), STL decomposition into trend/seasonal/residual components (Q7, Q8), moving-average smoothing and crossover analysis (Q9), and trend removal via differencing for business-cycle analysis (Q10). Both R and Python were used as specified per question, with simulated datasets constructed to realistically reflect each scenario so that the resulting graphs and interpretations are directly meaningful.